

Discussion 5 Hash

Discussion Topic:

How do hash tables function as search techniques in modern applications, and what are the key differences between perfect and non-perfect hashing approaches? Use a specific example to illustrate the practical benefits of hashing for data retrieval.

My Post:

Hello Class,

Hash tables are data structures that store unordered elements via hashing, mapping each element to a location in an array (Lysecky & Vahid, 2019). They use a hash function to mathematically compute search keys or hash keys from array element indexes. This allows hash tables to be very efficient for searching and consequently for inserting and removing elements, requiring only $O(1)$ average time in contrast to $O(N)$ linear search of a list or $O(\log N)$ for binary search.

Most hash functions use the modulo operator; the simplest manifestation of this is $\text{array_index} \% \text{number_of_items}$. Using the number_of_items is practical as it ensures that the hash key stays within the array's index bounds.

However, hash functions can map several indexes to the same hash key, creating collisions. To handle collisions, methods such as chaining or open addressing, including linear probing, quadratic probing, and double hashing, can be used. These methods are used when hash functions do not produce perfect hashing outcomes. The main difference between perfect and non-perfect hashing is how these approaches handle collisions. A perfect hash function gives each key a unique hash value, so collisions do not occur. By contrast, non-perfect hashing is faster but accepts that collisions are inevitable and requires a resolution strategy. This matters in real-world applications because identical keys from different can be generated by an hash function; for example, a simple hash based on the sum of character codes gives the same value for "hello world" and "world hello", moreover, even an improved weighted hash can still produce collisions such as "ad" and "ga" (CSU Global, n.d.). In other words, the uniqueness of the hash keys produced by a perfect hashing system is ideal, but in practice, most applications use non-perfect hashing plus a collision-resolution strategy.

A specific example that illustrates the practical benefits of hashing for data retrieval is managing active user sessions in web applications. In these applications, a session management system usually assigns a session ID that acts as the key that is associated with specific session data. When the user returns to the web page, the session manager, using the hash key, quickly retrieves the specific user session data without checking every user record one by one. For this application, most systems use non-perfect hashing, and because collisions can still occur, the systems also use strategies such as chaining or probing to handle collisions.

-Alex

References:

CSU Global. (n.d.). *Module 5: Hash Tables, Heaps, and Treaps* [Interactive lecture]. CSC506 - Design and Analysis of Algorithms. Canvas.

Lysecky, R., & Vahid, F. (2019). Module 5: Hash Tables, Heaps, and Treaps. *Data structures essentials: Pseudocode with Python examples*. zyBooks.